

Hardware Implementation of Stealthy and Lightweight Backdoor for CRYSTALS-Kyber

Suraj Mandal
Dept. of CSE
IIT Kanpur, India

Prasanna Ravi
Temasek Labs
NTU Singapore, Singapore

M Dhilipkumar
Dept. of CSE
IIT Kanpur, India

Debapriya Basu Roy
Dept. of CSE
IIT Kanpur, India

Anupam Chattopadhyay
College of Computing and Data Science
NTU Singapore, Singapore

Abstract—The threat of practical quantum attacks has catapulted viable alternatives like Post-Quantum Cryptography (PQC) into prominence. The adoption and integration of standardized PQC primitives across the entire digital stack are promoted by various standardization bodies, governments, and major corporate houses. A serious challenge in quantum migration is to ensure that there is no hidden backdoor in the PQC implementations of a hybrid cryptosystem (support for both pre-quantum and post-quantum algorithms), which are often procured from a third-party vendor. In this manuscript, we investigate the possibility of a kleptographic backdoor on the NIST-recommended key-encapsulation mechanism CRYSTALS-Kyber. The modified Kyber key-generation algorithm achieves indistinguishable decryption failure probability compared to the original CRYSTALS-Kyber. The kleptographic module is also implemented in FPGA, embedded inside the CRYSTALS-Kyber accelerator with a very low area overhead (283 LUTs or 2% of total area), and thus can easily pass performance and functionality tests.

Index Terms—ML-KEM, Kleptography, Backdoor, CRYSTALS-Kyber.

I. INTRODUCTION

Kleptography [1] is a deceptive technique to recover the secret through a backdoor embedded in a cryptographic system. In kleptography, the attacker uses a public key infrastructure to maintain the undetectability of the backdoor in such a way that the output of the modified backdoor-ed algorithm remains polynomially indistinguishable from the output of the original algorithm. Existing discrete logarithm-based cryptosystems like elliptic curve cryptography (ECC) [2], RSA [3], etc., have been proved to be vulnerable against kleptographic attack [4], [5]. In recent times, kleptographic constructions have been shown to be applicable to post-quantum cryptographic algorithms also [6]–[12], exploring the possibility of inserting a kleptographic backdoor in post-quantum cryptographic algorithms to leak the secret key or the message. Kleptographic attack is considered to be a malicious modification to the algorithm, and is different from vulnerabilities like hardware Trojans, which targets malicious modification of the crypto-implementations [13], [14]. A hardware Trojan circuit has a specific trigger and payload, where the malicious activity happens once the trigger is activated. Note that this malicious activity can be identified if we compare the malicious output with the original output, obtained from the golden reference implementation. On the other hand, even though the output of a kleptographic construction is insidious, it can be made statistically indistinguishable from the correct output, and hence poses a more severe threat than traditional hardware Trojans.

Figure 1 illustrates the mechanism of a kleptographic backdoor. One uses a black-box implementation of an algorithm B . The attacker pre-installs a backdoor in B using a public key algorithm P . A straightforward way for an attacker is to encrypt the secret key of B using the public key of P . This way, only the attacker can get the secret key of B from the output of algorithm B by using the secret key of P as the backdoor.

The threat of a kleptographic backdoor has been more critical due to the recommendation of a hybrid cryptosystem by organisations like NIST, National Cyber Security Centre (USA), Apple, etc [15]–[17]

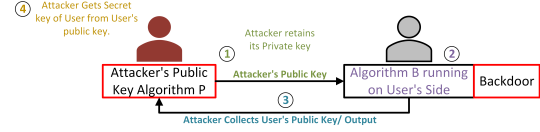


Fig. 1: Kleptographic Attack

to provide a safeguard for classical systems from quantum attacks. In [15], NIST also recommends adopting a post-quantum cryptographic (PQC) standard after the year 2035, replacing classical cryptosystems like ECC, RSA, etc. For a smooth transition, these organisations advocate the interim use of hybrid cryptosystems combining both pre-quantum and post-quantum algorithms to ensure adequate security. In the current scenario, a hybrid cryptosystem will provide support for NIST-recommended quantum secure key encapsulation mechanism (KEM), i.e., CRYSTALS-Kyber, and for a quantum secure DSA, CRYSTALS-Dilithium, along with support for elliptic curve DSA (ECDSA) for a pre-quantum public key infrastructure. Supporting two different public key infrastructures (pre-quantum and post-quantum) in a hybrid cryptosystem will further facilitate an adversary to install a kleptographic backdoor since, the backdoor generation logic can already be a part of the IP itself without any extra overhead.

In literature, the works that exhibit practical instantiations of kleptographic attack on Kyber [6], [12] provide software-based implementations using a pre-quantum or a post-quantum backdoor. However, in [12], the decryption failure probability for Kyber [18] has been increased due to the increased range of the error polynomial. The work [6] maintains the same error distribution as Kyber but uses 3 different central binomial distribution (CBD) functions for the generation of the secret key. In this work, we have tried to answer two critical questions in the context of kleptographic backdoor construction for Crystals-Kyber. The first question is: How to make a backdoor stealthy to a blackbox user?, and the second question is: How to insert the backdoor without performance overhead?.

How to make a backdoor stealthy to a blackbox user? An efficient backdoor should be designed in such a way that a blackbox user can not statistically distinguish between a correct output and a malicious output of the algorithm. We have shown that the backdoor in [12] can be discovered by a blackbox user by executing a verifiability test. Also, a blackbox user can run known answer tests (KAT) to ensure the correctness of the IP. Although the work of [6] is stealthier than [12], it will fail the KAT test. We have proposed two kleptographic backdoor constructions embedded in Module-Lattice Key Encapsulation Mechanism (ML-KEM) or CRYSTALS-Kyber's key-generation: i. The first backdoor construction uses ECC. ii. The second backdoor construction uses static values generated by ECC, which are only known to the adversary, along with AES to generate the randomness. Anyhow, AES-256 will be available in the cryptographic suite to facilitate symmetric key operation. Both of our proposed backdoors are stealthy, i.e., both of these methods provide the flexibility to

the user for verifying the KAT tests (debug mode) with a seed as input, as well as to generate random seeds with system randomness (execution mode) according to the algorithmic construction of Kyber. The backdoor has been incorporated into the execution mode that produces a malicious but polynomially indistinguishable output from the original Kyber's Key generation. The blackbox user will have no idea about the malicious output, as our backdoor will pass the KAT test in debug mode, and will take a random seed as input in execution mode with a polynomially indistinguishable output.

How to insert the backdoor without performance overhead? An efficient backdoor-ed algorithm should also have nearly similar performance overhead (latency in case of software implementation/hardware overhead and latency in terms of hardware implementation) like the original algorithm. The backdoor of [12] uses ECC, and [6] uses classic McEliece, which has a very high latency in terms of both hardware and software implementations. Our second backdoor construction ensures a negligible increment in latency compared to Kyber key generation and outperforms the work of [6], [12] in terms of performance overhead. We have also shown that the hardware overhead of our proposed backdoor is only 2% compared to the total overhead of Kyber key generation.

The contributions of our work are summarized below:

- We have proposed two kleptographic backdoors, the first construction uses an ECC to create the backdoor, while the second construction uses static values of ECC, which are only known to the adversary, along with AES to generate the randomness.
- Both of our proposed kleptographic constructions are stealthier than [12], and our second backdoor also outperforms [6] in terms of performance overhead.
- We have also provided a mathematical proof for the undetectability of our backdoors, even though the output is malicious.
- We present a hardware implementation of such a kleptographic backdoor on the FPGA platform, where the backdoor is incorporated with a very low area overhead. To the best of our knowledge, this is the first work that shows a kleptographic backdoor inserted into a Hardware IP.

The rest of the paper is organised as follows. Section II provides a brief introduction to CRYSTALS-Kyber and samplers used in Kyber. In Section III, we have described the state-of-the-art works in kleptographic backdoors on PQC algorithms. Section IV provides a detailed analysis and the drawbacks of the existing most relevant two backdoors. Our proposed kleptographic attack has been described in Section V. The hardware for such a backdoor, combining the kleptographic attack inside a hybrid cryptosystem, has been illustrated in Section VI. Finally, in Section VII, we conclude the paper.

II. BACKGROUND

In this section, we will briefly describe the key-generation algorithm of CRYSTALS-Kyber and the CBD function of Kyber as we need to modify it for backdoor implementation.

A. CRYSTALS-Kyber

CRYSTALS-Kyber or ML-KEM is a CCA2-secure KEM [19] and is defined over a quotient ring $Z_q[x]/(X^n + 1)$ where q is the prime modulus and n is the polynomial degree. Depending on different parameter values, CRYSTALS-Kyber has three variants, i.e., Kyber-512, Kyber-768, and Kyber-1024. Kyber KEM consists of three functions: Key-Generation, Encapsulation, and Decapsulation. As our backdoor is installed in the key-generation procedure, we have described the functionality of key-generation in detail as given below:

- **Key-Generation():** Key generation function in public key encryption (PKE) creates a public key/private key pair. An algorithmic overview of the Key-Generation() of Kyber function has been shown in Algorithm 1 where the public key has been denoted by pk and the private key has been denoted by sk . Vector A ($\mathcal{R}_q^{k \times k}$) is directly generated in the number theoretic transformation (NTT) domain and with rejection sampling. Vector s ($\mathcal{R}_q^k$) and vector e ($\mathcal{R}_q^k$) are generated by random sampling with the central binomial distribution (CBD) where the error distribution varies from $-\eta$ to η . After converting s and e into NTT domain, polynomial $\hat{t}$ is generated by computing $\hat{A} \times NTT(s) + NTT(e)$. Thus, vector s becomes the secret key, and vector $(seed_A || \hat{t})$ becomes the public key where $seed_A$ is a random number sampled from a hash function and is used to generate the vector A . $G()$, $XOF()$ and PRF functions are different instantiations of Keccak [20].

Algorithm 1 Key Generation Procedure of IND-CPA secure CRYSTALS-Kyber

```

1: procedure KeyGen
2:    $seed_A \in \mathcal{B}^{32} \leftarrow \text{Sample}_U()$   $\triangleright$  Generated using  $G()$ 
3:    $seed_B \in \mathcal{B}^{32} \leftarrow \text{Sample}_U()$   $\triangleright$  Generated using  $G()$ 
4:    $\hat{A} \in \mathcal{R}_q^{k \times k} \leftarrow \text{Expand}(seed_A)$   $\triangleright$  Sample  $\hat{A}$ , Uses XOF()
   and Parse()
5:    $s \in \mathcal{R}_q^k \leftarrow \text{Sample}_B(seed_B, coin_s)$   $\triangleright$  Sample  $s$ , Uses
   PRF() and CBD()
6:    $e \in \mathcal{R}_q^k \leftarrow \text{Sample}_B(seed_B, coin_e)$   $\triangleright$  Sample  $e$ , Uses
   PRF() and CBD()
7:    $\hat{t} = \hat{A} \circ NTT(s) + NTT(e)$   $\triangleright$  Compute  $NTT(\hat{t})$ 
8:   Return  $(pk = (seed_A, \hat{t}), sk = (NTT(s)))$ 
9: end procedure

```

- **Samplers:** Polynomial A in Algorithm 1 is generated with rejection sampling (Parse), whereas secret (s) and error (e) polynomials are generated from CBD. The functionality of CBD can be defined as $\text{Sample}(a_1, a_2, \dots, a_\eta, b_1, b_2, \dots, b_\eta) \leftarrow \{0, 1\}^{2\eta}$ with output $\sum_{i=1}^{\eta} (a_i - b_i)$. η here decides the error range ($[-\eta, +\eta]$) for the output of CBD sampling. The values of η used in three Kyber-512, Kyber-768 and Kyber-1024 are 3, 2 and 2 respectively.

III. RELATED WORKS

In this section, we will briefly describe the existing kleptographic attacks on PQC algorithms. In [7], the authors proposed an ECC-based backdoor for the NTRU public key encryption (PKE) algorithm. Later on the work [8] shows a generic idea of how a Ring-LWE-based encryption scheme be converted to a klepto scheme. In another work [9], the authors proposed a backdoor for NTRU PKE encryption without the involvement of any other algorithm. In [10] the authors show a kleptographic attack on generic Ring-LWE encryption. The work [11] recovers a 32-byte message by incorporating a backdoor. The most recent two works [6], [12] proposed kleptographic attacks on ML-KEM i.e. scalable to recover any message size. Only works that explore backdoors for ML-KEM are [12] and [6]. We have provided a brief overview of the work [12] in Section III-A as our backdoor follows a similar ECC-based backdoor construction. The work [6] uses the classic McEliece (Attacker's PKE) algorithm for generating the ciphertext to be embedded with the error vector by encrypting the public key of the attacker with the session key K . The seed for public and error vectors in [6] is generated by session key K .

Some of the works in the literature have explored Trojan-based attacks on post-quantum algorithms [13], [14]. In both of these works

[13], [14], a user can observe the incorrect output of the decapsulation or signing as the output is malicious for the invocation of a Trojan. In contrast, our work imposes a more severe threat as a user will not be able to statistically distinguish between correct and malicious output, yet the attacker will get the secret.

A. Kleptographic Attack of [12]

The kleptographic construction of [12] is based on public key infrastructure. The attacker generates a backdoored secret key sk_{bd} and a public key pk_{bd} offline with his/her PKE. pk_{bd} is then installed in the key generation of the target cryptosystem, i.e., Kyber-768. The targeted key generation algorithm of Kyber (Algorithm 1) needs to generate vectors A , s and e from $seed_A$ and $seed_B$. In the backdoor-ed key generation algorithm of [12], the attacker

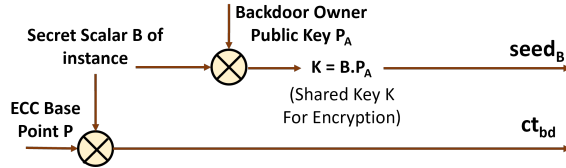


Fig. 2: Modified Seed Generation for Secret Key s in [12]

provides a modified algorithm to the user, where $seed_B$ is generated with the help of the attacker's public key pk_{bd} , incorporating some internal randomness as shown in Figure 2. After generating the seeds, $\hat{A} \times NTT(s) + NTT(e)$ is computed. Now, to recover the secret, the attacker hides additional information with $\hat{A} \times NTT(s) + NTT(e)$. The modified key-generation function has the following steps:

- A function $Enc_{bd}()$ is executed for generation of ct_{bd} and $m_{bd}(seed_B)$ where $ct_{bd} = S_c.P$, $m_{bd} = S_c.pk_{bd}$. Please note that P is the base point of the CURVE K-409 i.e. $pk_{bd} = B.P$ where B is a randomly sampled scalar.
- As the size ct_{bd} is 832-bit long and Kyber-768 has 768 coefficients in $\hat{A} \times s + e$, the attacker can hide 2-bits per coefficient.
- To hide the ct_{bd} in the public key, and then to recover easily, the backdoor installed by the attacker computes a compensation h for each of the two bits of the ct_{bd} value, such that h follows the correct error range.

The attacker now retrieves the public key of key-generation $pk = NTT(\hat{A} \times s + e + h)$ and recovers ct_{bd} with its secret key recovery function. Eventually, with the attacker's secret key sk_{bd} , the attacker successfully recovers $m_{bd}(seed_B)$ i.e. seed to the secret of the user's key-generation function. The work [12] thus successfully recovers the secret key of the target key generation function.

IV. ANALYSIS OF EXISTING BACKDOORS

In this section, we shall provide a detailed analysis about the existing kleptographic attacks as well as we shall point out the shortcomings of these works.

A. Analysis of ECC based Kleptographic Attack ([12])

The attack in [12] creates a backdoor for Kyber-768 by public key encryption using CURVE K-409. The work added an additional error to the CBD output of the key generation function that resulted in the total error distribution as $[-4, 4]$. Eventually, the decryption failure probability drops to $2^{-124.5}$, whereas the decryption failure probability for the original Kyber is $\approx 2^{-165.2}$. Although checking the difference between failure probabilities is not a feasible task.

Another major concern of the work [12] is that if a user checks for the error distribution plot by calculating $t - A \times s$, where $t = A \times s + e$

is the public vector and seed of A is also public, the user can clearly observe the presence of errors ranging from -4 to $+4$ instead of -2 to $+2$ by observing a single error polynomial and thus the user can guess the presence of a backdoor.

B. Analysis of Kleptographic Attack in ([6])

The work [6] uses three different samplers along with a random number generator that produces a random number within the range 0 to 100 to generate the coefficients of e , which adds extra overhead to the implementation. During the key recovery, the attacker brute forces for few bit positions depending on the boundary condition of the public vector t . Also, the backdoor in [6] will not pass the KAT test as the backdoor is active all the time.

Although the work [6] has given proof of undetectability, there is a flaw in their open-sourced C-code that causes a slightly biased error distribution compared to the original Kyber. We have collected a dataset of 100k of such error polynomials of [6] and have plotted a histogram of $\{-2, -1, 0, 1, 2\}$ as shown in Figure 3a. We can find a clear distinction between the count of 2 and -2 , whereas in the original Kyber, the count of 2 and -2 is the same. However, we have proposed a code fix in section IV-C such that the error distribution of [6] becomes indistinguishable from the original Kyber.

C. Fixing the Code of Kleptographic backdoor of [6]

The CBD function in [6] takes input distributions $D_1 = \{-1 : \frac{1}{2}, 1 : \frac{1}{2}\}$ or $D_0 = \{-2 : \frac{1}{8}, 0 : \frac{3}{4}, 2 : \frac{1}{8}\}$ or $B_2 = \{-2 : \frac{1}{16}, -1 : \frac{1}{4}, 0 : \frac{3}{8}, 1 : \frac{1}{4}, 2 : \frac{1}{16}\}$. Based on a random integer $temp$ ($0 \leq temp \leq 100$), the output is chosen according to the cumulative distribution; e.g., for D_0 and $temp = 54$, the output is 0 ($(\frac{1}{8} \times 100) < 54 \leq (\frac{100}{8} + \frac{3}{4} \times 100)$). However, since $temp$ is an integer, the probability ranges become imprecise. For instance, in D_0 , -2 should occur when $temp \leq 12.5$, but only integers 0–12 are valid, slightly skewing the distribution (Figure 3a).

To resolve this, we use $temp \in [0, 128]$, ensuring clean integer boundaries for all distribution intervals. After this fix, a new set of 100k samples shows that frequencies of -2 and 2 now align closely with the original Kyber distribution (Figure 3b).

V. PROPOSED KLEPTOGRAPHIC CONSTRUCTION

In this section, we will describe our threat model, followed by the backdoor insertion in the key-generation procedure of Kyber.

A. Threat Model

In this paper, we have considered a hybrid cryptosystem with support for both pre-quantum and post-quantum algorithms. The designer provides a hardware accelerator/software code that includes quantum secure KEM, i.e., CRYSTALS-Kyber, and support for ECDSA for pre-quantum PKE as well as AES-256 for supporting symmetric key operations to the user. The adversary and the designer are the same person in our threat model. The deliverable hardware accelerator/software core will have debug mode and execution mode (deployment mode) as mentioned in Section I.

In the threat model (Figure 4), in the Gen_{bd} stage, the attacker runs its keygen function and computes his/her secret key (sk_{bd}) and public key (pk_{bd}) in offline mode. He/she installs the pk_{bd} in the developed hybrid crypto-system and supplies it to the user. The user runs the modified kleptographic keygen function of PQC-KEM with the installed pk_{bd} and generates his public key pk and secret key sk . The adversary will only need to access the output of key generation (public key pk) of PQC-KEM. After retaining pk , the adversary runs its' private decryption function to get the user's secret key sk .

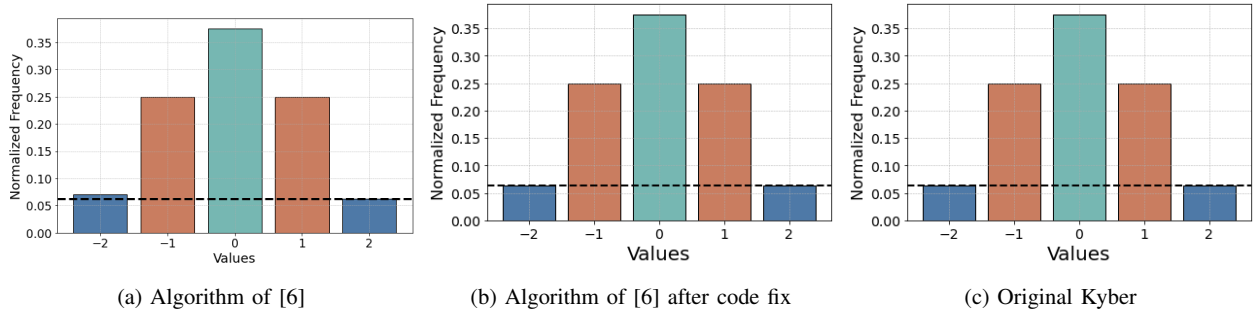


Fig. 3: Error Distribution Plot of [6] with and without Code Fix

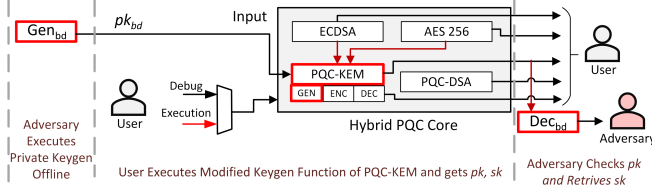


Fig. 4: Threat Model

B. Proposed Kleptographic Construction I

We will now describe our first kleptographic construction, which is an improved version of the kleptographic backdoor of [12]. The adversary will generate its public key pk_{bd} (seed of the backdoor) and private key sk_{bd} by running the KeyGen function of CURVE-25519 offline. The attacker has modified the seed for the generation of s and e in such a way that it will be easily recoverable to the attacker by only observing the output of the backdoor-ed key generation. The working principle of our kleptographic key-generation has been described below:

- The backdoored key generation function generates the ct_{bd} and m_{bd} using the similar procedure as [12]. The detailed method of generating ct_{bd} and m_{bd} has been described in Section III-A.
- m_{bd} is considered as the seed ($seed_B$) in execution mode for generation of secret key s and error vector e with modified CBD function that generates output in range $[-1, +1]$.
- The error polynomial ct_{bd} has also been generated which should be hidden in the public key pk and will help the attacker to find m_{bd} . The straightforward way would be adding the error polynomial ct_{bd} alongwith t . But that can completely hide the new error polynomial ct_{bd} . We have added a compensation h with t such that the adversary can easily find out ct_{bd} from t . The compensation h has been added to t (Algorithm 2) such that $t' = t + h = p[i] \bmod c$ where $c = \lceil b/(k \cdot n) \rceil$. Please note that we have added the h in such way so that total error range of public key pk remains same as original Kyber.

Although the methodology looks similar to the algorithm of [12], we have improved the attack in such a way that it would be extremely difficult for a user to detect the presence of a backdoor even if he/she plot the error distribution curve. We have listed the major changes in our algorithm that make it superior to the backdoor generation algorithm of [12] below:

- We have used CURVE-25519 as adversary's PKE (Line number 7 of Algorithm 2), which is a very popular curve for ECDSA. To maintain the error distribution as the original Kyber, we gener-

ated a 765-bit ciphertext from CURVE-25519 by incorporating y coordinate [21].

- Following the changes in line number 7, $seed_B$ and ct_{bd} (seed for s and e) has been generated by running the attacker's PKE function $Enc_{bd}()$ along with by generating some internal randomness where ct_{bd} is the added error polynomial to $\hat{A} \times s + e$.
- To keep the decryption failure probability the same as Kyber, we have applied two modifications to the algorithm. The first change is that we have modified the CBD functions (Line number 15 – 21 of Algorithm 2) such that it generates error coefficients in range $[-1, +1]$ (η_E) in execution mode whereas in debug mode the error coefficients will be generated in range $[-2, +2]$ (η_D). The second modification lies in adding the error generated from ct_{bd} . As we are using public key of curve-25519 as ct_{bd} , we have added 1-bit of error with 765 distinct coefficients of Kyber-768 whereas in [12], the error coefficients were in the range $[-2, +2]$. With these aforesaid modifications, we are able to maintain the total error distribution of key-generation between $[-2, +2]$, whereas the total error distribution of [12] was in the range $[-4, +4]$. A theoretical proof for the error distribution of our backdoor has been shown in Section V-D. We have collected 100k sample data to test the error distribution in our improved kleptographic key-generation algorithm. In Figure 5, we have plotted the error distribution by calculating the difference $t' - (\hat{a}s)$ of Algorithm 2 for 100k different inputs. Figure 5 clearly shows that our modified backdoor perfectly aligns with the error distribution plot of the original Kyber, whereas the error distribution plot of [12] differs a lot from the original Kyber. This way, a user can detect the kleptographic backdoor in [12] (as the value of $seed_A$ is public, the user can compute $t' - \hat{A}s$), whereas our plotted error distribution is indistinguishable from the original Kyber.
- To retrieve the secret key, there is a slight change in the secret key recovery algorithm of the attacker, i.e., in line number 43 of Algorithm 2, we have used the mod value as 2.

C. Proposed Kleptographic Construction II

Our second kleptographic construction has exactly the same algorithmic construction. The only difference lies in generating the random seeds from the $Enc()$ function (Line no. 7 of Algorithm 2) and the corresponding $Dec()$ (Line no. 46 of Algorithm 2) function on attacker's end. In our previous construction (Algorithm 2), the $Enc()$ function is similar to the work [12] as shown in Figure 6. But there are certain disadvantages of this proposed method. In $Enc()$, key generation involves two ECC scalar multiplications, which are significantly more costly than Kyber's key generation. In [6], the authors have also used Classic-Mceliece encryption function for backdooring that has a much higher latency than the Kyber key

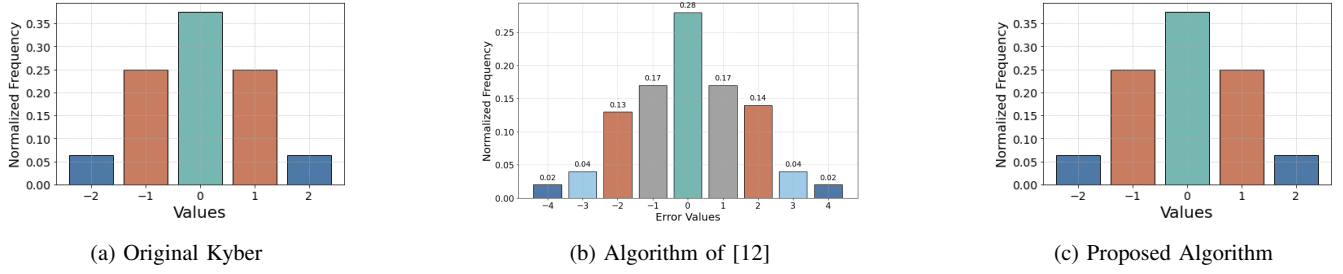


Fig. 5: Error Distribution Plot for Key Generation of Kyber

Algorithm 2 Our Modified Kleptographic Attack on Kyber KEM

```

1: procedure BACKDOOR INSTALLATION (OFFLINE)
2:    $(pk_{bd}, sk_{bd}) = \text{KeyGen}_{bd}()$ 
3:   Install  $pk_{bd}$  on target, retain  $sk_{bd}$  offline
4: end procedure

5: procedure BACKDOOR-ED KEY GENERATION:  $C'(pk_{bd}, seed_{debug}, mode)$ 
6:    $\eta_E = 1, \eta_D = 2$ 
7:    $Enc_{bd}(pk_{bd}) = (ct_{bd}, m_{bd})$   $\triangleright$  Backdoor Encryption
8:    $seed_A \in \mathcal{B}^{32} \leftarrow \text{Sample}_U()$ 
9:   if  $mode == 'E'$  then
10:     $seed_B \in \mathcal{B}^{32} = m_{bd}$   $\triangleright m_{bd}$  used as  $seed_B$ 
11:  else
12:     $seed_B \in \mathcal{B}^{32} = seed_{debug}$   $\triangleright pk_{bd}$  used as  $seed_B$ 
13:  end if
14:   $\hat{a} \in R_q^{k \times k} \leftarrow \text{Expand}(seed_A)$   $\triangleright$  Sample  $\hat{a}$ 
15:  if  $mode == 'E'$  then
16:     $s \in R_q^k \leftarrow \text{Sample}_B(seed_B, coin_s, \eta_E)$   $\triangleright$  Sample  $s$ 
17:     $e \in R_q^k \leftarrow \text{Sample}_B(seed_B, coin_e, \eta_E)$   $\triangleright$  Sample  $e$ 
18:  else
19:     $s \in R_q^k \leftarrow \text{Sample}_B(seed_B, coin_s, \eta_D)$   $\triangleright$  Sample  $s$ 
20:     $e \in R_q^k \leftarrow \text{Sample}_B(seed_B, coin_e, \eta_D)$   $\triangleright$  Sample  $e$ 
21:  end if
22:   $t = \text{INTT}(\hat{a} \circ \text{NTT}(s)) + e$ 
23:   $ct_{bd} = (bt_0, bt_1, \dots, bt_b); c = \lceil b/(k \cdot n) \rceil; p = 0; h = 0$ 
24:  for  $i$  from 0 to  $\lceil b/c \rceil - 1$  do
25:     $p[i] = bt_j$   $\triangleright$  Compute Encoded Payload  $p$ 
26:  end for
27:  for  $i$  from 0 to  $\lceil b/c \rceil - 1$  do
28:     $h[i] = \text{Compute\_Compensation}(t[i], p[i])$   $\triangleright$  Compute Compensation  $h$ 
29:  end for
30:  if  $mode == 'E'$  then
31:     $t' = t + h$   $\triangleright$  Adding Compensation  $h$  to  $t$ 
32:  else
33:     $t' = t$ 
34:  end if
35:  Return  $pk = (seed_A, \hat{t}' = \text{NTT}(t'))$ 
36: end procedure

37: procedure SECRET KEY RECOVERY( $pk = (seed_A, \hat{t}')$ )
38:   $t' = \text{INTT}(\hat{t}'), p = 0$ 
39:  for  $i$  from 0 to  $\lceil b/c \rceil - 1$  do
40:     $p[i] = t'[i] \bmod 2$   $\triangleright$  Computing encoded payload  $p$  from  $t'$ 
41:  end for
42:   $ct_{bd} = \text{Reconstruct}(p)$   $\triangleright$  Recovering payload  $ct_{bd}$ 
43:   $m'_{bd} = \text{Dec}_{bd}(sk_{bd}, ct_{bd})$   $\triangleright$  Backdoor Decryption
44:   $s \in R_q^k \leftarrow \text{Sample}_B(m'_{bd}, coin_s)$   $\triangleright$  Recover secret module  $s$ 
45:  Return  $s$ 
46: end procedure

```

generation. This may lead the user to detect the backdoor in terms of latency. To overcome this limitation, we have reconstructed the $Enc()$ in such a way that the user only needs to execute one XOR and two AES operations with a negligible latency as shown in Figure 7. The rest of the algorithm (Algorithm 2) remains the same. The modified $Enc()$ (Figure 7) has been explained below:

- Attacker picks a static scalar B and a static plaintext P_{text} , that are known only to the attacker. He/She then installs two fixed values $ctemp_{bd} = B.P$ and $seed_{bd} = B.P_A$, where P is the base point of ECC and P_A is the public key of the attacker.

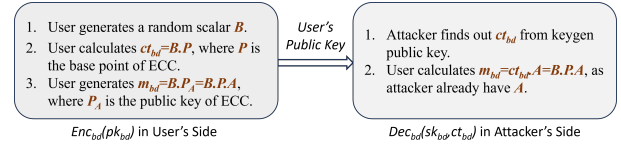


Fig. 6: $Enc_{bd}()$ and $Dec_{bd}()$ functions of Algorithm 2

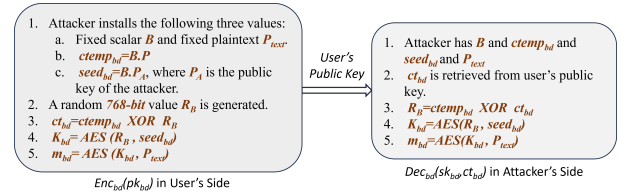


Fig. 7: Modified $Enc_{bd}()$ and $Dec_{bd}()$ functions of Algorithm 2

- Now a 768-bit random value R_B is generated and xor-ed with $ctemp_{bd} = B.P$ for generating ct_{bd} . XOR-ing with a random number ensures the freshness of ct_{bd} in each execution.
- To generate the seed (m_{bd}) for s and e , two consecutive AES encryption are performed. First AES encryption has been performed on $seed_{bd}$, keeping R_B as the key of AES. The second AES operation takes P_{text} as input, with the first AES output as the key.

For key recovery, as the attacker has the values of B and P_{text} , he/she can generate $ctemp_{bd}$ and ct_{bd} will be computed by the same procedure as shown from Line no. 40 to Line no. 45 of Algorithm 2. Then, the attacker can find out R_B by xor-ing $ctemp_{bd}$ and ct_{bd} . As the values of $seed_{bd}$ and P_{text} are already available to the attacker, m_{bd} can be found out easily with two consecutive AES encryptions.

TABLE I. Different values of $a - b$ depending on a and b

a	0	0	1	1
b	0	1	0	1
$a - b$	0	-1	1	0

D. Proof of Undetectability of the Proposed Backdoors

Our work adds two error polynomials with coefficient values $\{-1, 0, 1\}$ and generates the exact same error distribution as Kyber with coefficient values $\{-2, -1, 0, 1, 2\}$. In this section, we have provided a theoretical analysis that proves the undetectability of the proposed backdoor. Unlike CBD function of the original Kyber, where the least significant two bits have been considered for the generation of one coefficient, we have used the least significant one bit to generate one coefficient. Thus, our modified CBD function generates coefficients in the range $\{-1, 0, 1\}$ as shown in Table I.

TABLE II. Probability Distribution of Our Final Error Polynomial

	$P(e'[i] = -2)$	$P(e'[i] = -1)$	$P(e'[i] = 0)$	$P(e'[i] = 1)$	$P(e'[i] = 2)$
Ways	$(h[i] = -1, e[i] = -1)$	$(h[i] = -1, e[i] = 0)$ $(h[i] = 0, e[i] = -1)$	$(h[i] = -1, e[i] = 1)$ $(h[i] = 0, e[i] = 0)$ $(h[i] = 1, e[i] = -1)$	$(h[i] = 1, e[i] = 0)$ $(h[i] = 0, e[i] = 1)$	$(h[i] = 1, e[i] = 1)$
Total Probability	$1/4 * 1/4$ $1/16$	$1/4 * 1/2 + 1/2 * 1/4$ $1/4$	$1/4 * 1/4 + 1/2 * 1/2 + 1/4 * 1/4$ $3/8$	$1/2 * 1/4 + 1/4 * 1/2$ $1/4$	$1/4 * 1/4$ $1/16$

TABLE III. Comparison b/w existing Kleptographic Constructions

	[6]	[12]	Our Proposed Backdoor I	Our Proposed Backdoor II
Error Distribution	Indistinguishable	Detectable	Indistinguishable	Indistinguishable
Number of CBD functions	3	2	2	2
Debug/Execution mode	No	No	Yes	Yes
Passes KAT test	No	No	Yes	Yes
Uses PKE	Yes	Yes	Yes	No

Hence, from Table I probability of occurrence of -1 , 0 and 1 becomes $\frac{1}{4}$, $\frac{1}{2}$ and $\frac{1}{4}$ respectively. Another error $h[i]$ (line number 30 of Algorithm 2) has been calculated by subtracting the $P[i]$ (line number 26 of Algorithm 2) from the least significant bit of $t[i]$ (line number 22 of Algorithm 2). Being an one-bit subtraction result, $h[i]$ also follows the same error distribution as our CBD function. After adding error coefficients $h[i]$ with e , the coefficients of the new error polynomial e' will range in $\{-2, -1, 0, 1, 2\}$ with a probability distribution $\{-2 : \frac{1}{16}, -1 : \frac{1}{4}, 0 : \frac{3}{8}, 1 : \frac{1}{4}, 2 : \frac{1}{16}\}$ indistinguishable to the error distribution of original Kyber. We have shown the calculation for the probability distribution of the new error polynomial e' in Table II.

E. Comparison between [12], [6] and the Proposed Backdoors

In this section, we have provided a comparison analysis among [6], [12], and our work. In a nutshell, the differences have been pointed out in Table V. The error distribution plot of the proposed backdoor, as well as the backdoor of the work [6] after our proposed code fix, matches with the original Kyber, keeping the decryption probability the same. In contrast, the backdoor of [12] is detectable easily by observing the error distribution plot.

In debug mode, the backdoors of our work will qualify the KAT tests, and in execution mode, the backdoor is activated, whereas, in the existing works [6], [12], the backdoors do not have this feature. Thus, the works [6], [12] fail to qualify the KAT tests as the backdoor is an integrated part of the key-generation algorithm itself.

Our backdoor needs two different CBD functions, whereas the work [6] needs three different CBD functions. The CBD functions of [6] also need a random number for the generation of each coefficient. The random number generation not only adds to the latency but also needs additional hardware overhead. In Section VI, we discuss the implementation of our kleptographic module. Another drawback of the works [6], [12] is that these backdoors consume a significant latency over the original Kyber for using PKE. As reported in [6], the software implementation of backdoored key generation incurs an approximate 307% increase in latency. This may lead the user to detect the presence of a backdoor in the software IP. In contrast, the hardware implementation of our second backdoor construction has an increment of 4% in terms of latency compared to the original Kyber, which will be unnoticeable to the user. We have provided a complete area overhead and latency count in Section VI.

VI. HARDWARE IMPLEMENTATION OF KYBER KEY-GENERATION WITH HYBRID CRYPTOSYSTEM

We have implemented a programmable accelerator with dedicated instruction-set architecture (ISA) for key generation of Kyber-768

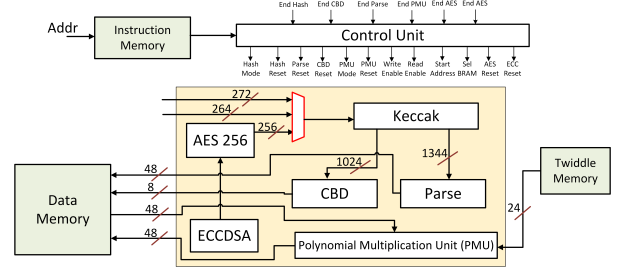


Fig. 8: Modified Kyber Key-Generation Architecture

integrated with the hybrid PQC architecture as shown in Figure 8. The complete architecture consists of instruction memory, data memory (memory for A, s, e, and ECDSA), twiddle memory, and different hardware modules like Keccak, CBD, parse, and polynomial multiplication unit (PMU), AES, and implementation of CURVE25519.

A. Kyber Keygen Architecture in Kleptographic Attack Mode

As discussed earlier our hybrid architecture provides support for PQC-KEM CRYSTALS-Kyber as well as support for pre-quantum algorithms like ECDSA and AES. Figure 8 provides a high-level architecture of Kyber keygen in execution or kleptographic attack mode. In debug mode, the user provides the initial seed to verify the known answer tests. In the execution mode, ECDSA using CURVE25519 or AES is used to generate the initial seed for Kyber key-generation. The value of η for CBD function is also selected depending upon the mode. As AES and CURVE25519 are already part of the hybrid cryptosystem, payload of the modified keygen algorithm is 4 multiplexers i.e. one 1-bit multiplexer to select the mode, one multiplexer of 256-bit for choice of initial seed, one 3-bit multiplexer to select the correct η for CBD and one 24-bit multiplexer to select the output between t and $t + h$ as shown in Algorithm 2.

TABLE IV. Hardware Overhead of Individual Modules

	LUT	FFs	DSP	BRAM
Keccak	2788	5628	0	0
PMU	935	918	2	14.5
CBD	537	1059	0	0
Parse	1379	1723	0	0
AES-256	3304	4114	0	0
CURVE25519	4567	4632	40	0

B. Hardware Implementation Results

As in this work, our goal is to estimate the extra overhead and latency due to a kleptographic backdoor in a hybrid cryptosystem. Therefore, for this work, we have not compared our work to the state-of-the-art implementations of CRYSTALS-Kyber. We have used Pynq-Z2 FPGA board (*xc7z020clg400-1*) for the implementation of the kleptographic hardware. The overhead of the individual hardware module in the kleptographic hardware is provided in Table IV.

Therefore, the proposed Kyber Keygen architecture in a normal setting has an overhead of 6871 LUTs, 8205 FFs, 2 DSPs and 12 BRAMs with a frequency of 209 MHz and completes the key

generation with a latency of 6408 clock cycles. With additional muxes for kleptographic attack, modified key generation consumes 7154 LUTs, 8205 FFs, 2 DSPs and 12 with the same frequency. Combining the AES and ECDSA (We have used same hardware for ECDSA as in [22]) the total overhead of our hybrid cryptosystem with kleptographic key generation is 15025 LUTs, 16951 FFs, 42 DSPs and 21 BRAMS with a frequency of 180MHz (same as frequency of the ECC module as the ECC module achieves the lowest frequency). Table V, shows that changes in the hardware overhead due to the incorporation of kleptographic attack is very low compared to the total overhead. Although our first backdoor construction has a higher latency, our second construction only consumes an additional 278 (~ 4%) clock cycles than the original Kyber key-generation.

TABLE V. Comparison between kleptographic keygen with hybrid cryptosystem and keygen in normal settings

	LUT	FF	DSP	BRAM	Frequency (Mhz)
Hybrid Cryptosystem Without Backdoor	14742	21670	42	21	180
Hybrid Cryptosystem With Backdoor	15025	21670	42	21	180

VII. CONCLUSION

In this work, we have successfully incorporated backdoor in a Kyber key generation such that the decryption failure rate remains indistinguishable from the original Kyber. Our improved kleptographic attack also shows that the user will not be able to detect the backdoor even if he/she plots the error distribution of the public key/secret key. We also have implemented our kleptographic algorithm in hardware and have shown that in a hybrid cryptosystem, an adversary can easily install a backdoor with very little performance overhead.

REFERENCES

- [1] A. Young and M. Yung, "Kleptography: Using cryptography against cryptography," in *Advances in Cryptology—EUROCRYPT'97: International Conference on the Theory and Application of Cryptographic Techniques Konstanz, Germany, May 11–15, 1997 Proceedings* 16. Springer, 1997, pp. 62–74.
- [2] F. PUB, "Digital signature standard (dss)," *Fips pub*, pp. 186–192, 2000.
- [3] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [4] A. Young and M. Yung, "The prevalence of kleptographic attacks on discrete-log based cryptosystems," in *Advances in Cryptology—CRYPTO'97: 17th Annual International Cryptology Conference Santa Barbara, California, USA August 17–21, 1997 Proceedings* 17. Springer, 1997, pp. 264–276.
- [5] —, "Cryptography as an attack technology: Proving the rsa/factoring kleptographic attack," *The New Codebreakers: Essays Dedicated to David Kahn on the Occasion of His 85th Birthday*, pp. 243–255, 2016.
- [6] W. Xia, G. Wang, and D. Gu, "Post-quantum backdoor for kyber-kem," in *Selected Areas in Cryptography*, vol. 2024, 2024.
- [7] R. Kwant, T. Lange, and K. Thissen, "Lattice klepto: Turning post-quantum crypto against itself," in *International Conference on Selected Areas in Cryptography*. Springer, 2017, pp. 336–354.
- [8] D. Xiao and Y. Yu, "Klepto for ring-lwe encryption," *The Computer Journal*, vol. 61, no. 8, pp. 1228–1239, 2018.
- [9] Z. Yang, R. Chen, C. Li, L. Qu, and G. Yang, "On the security of lwe cryptosystem against subversion attacks," *The Computer Journal*, vol. 63, no. 4, pp. 495–507, 2020.
- [10] Z. Yang, T. Xie, and Y. Pan, "Lattice klepto revisited," in *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, 2020, pp. 867–873.
- [11] T. Hemmert, "How to backdoor lwe-like cryptosystems," *Cryptology ePrint Archive*, 2022.
- [12] P. Ravi, S. Bhasin, A. Chattopadhyay, A. Aikata, and S. Sinha Roy, "Backdooring post-quantum cryptography: Kleptographic attacks on lattice-based kems," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, 2024, pp. 216–221.
- [13] P. Ravi, S. Deb, A. Baksi, A. Chattopadhyay, S. Bhasin, and A. Mendelson, "On threat of hardware trojan to post-quantum lattice-based schemes: A key recovery attack on saber and beyond," in *International Conference on Security, Privacy, and Applied Cryptography Engineering*. Springer, 2021, pp. 81–103.
- [14] S. Pagliarini, A. Aikata, M. Imran, and S. Sinha Roy, "Repqc: Reverse engineering and backdooring hardware accelerators for post-quantum cryptography," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024, pp. 533–547.
- [15] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, "Transition to post-quantum cryptography standards," National Institute of Standards and Technology, Tech. Rep., 2024.
- [16] "Next steps in preparing for post-quantum cryptography - ncsc.gov.uk," (Accessed on 10/24/2024). [Online]. Available: https://www.ncsc.gov.uk/whitepaper/next-steps-preparing-for-post-quantum-cryptography#section_5
- [17] "Blog - imessage with pq3: The new state of the art in quantum-secure messaging at scale - apple security research," (Accessed on 10/24/2024). [Online]. Available: <https://security.apple.com/blog/imessage-pq3/>
- [18] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "Crystals-kyber," *NIST, Tech. Rep*, 2017.
- [19] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "Crystals-kyber: a cca-secure module-lattice-based kem," in *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2018, pp. 353–367.
- [20] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "Keccak," in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2013, pp. 313–314.
- [21] C. Costello and B. Smith, "Montgomery curves and their arithmetic: The case of large characteristic fields," *Journal of Cryptographic Engineering*, vol. 8, no. 3, pp. 227–240, 2018.
- [22] D. B. Roy and D. Mukhopadhyay, "High-speed implementation of ecc scalar multiplication in gf (p) for generic montgomery curves," *IEEE transactions on very large scale integration (VLSI) systems*, vol. 27, no. 7, pp. 1587–1600, 2019.